ASSIGNMENT 2

Design C++ programs by applying functions and object-oriented concepts –function prototyping, call by reference, return by reference, inline functions, defaultarguments, function overloading, specifying a class, member functions, access specifiers,arrays within a class, static data members and member functions, arrays of objects and friend functions.

Subject: Object Oriented Programming using C++
CO1: Apply object-oriented programming principles and C++ constructs (data types, operators and control structures) to design and write structured OOP-based programs
Total Marks: 50 Nature: Individual Bloom's Level: K5 – Evaluate

Student Name	NELLEPALLI VINAY YADAV
Register Number	192472061
Department / Section	CSE-AI
Submission Date	11/11/2026
GitHub Repository Link	https://github.com/VINAYYADAV10/DSA0117---OBJECT-ORIENTED-PROGRAMMING-WITH-C-
Compiler used	g++ (Ubuntu 13.3.0) 13.3.0
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file

Contents of this report: Task 1 to Task 12 (analysis, alternative concept and complexity analysis, pseudocode, UML, implementation code, execution screenshots and test tables).

Q1: Call by Value, by Reference and by Pointer

1. Problem analysis

The hostel concession function prints the correct reduced fee inside the function but the fee stored in the student record never changes. The function received **a copy** of the fee, so only the copy was reduced. The concessions are merit 10 %, sports 15 % and single parent 25 %. All arithmetic uses integers, so the result of fee * percent / 100 is truncated.

2. Implementation – the function written three ways

Source: q1_three_ways.cpp

```
#include <iostream>
#include <chrono>

// 1. call by value : parameter is a copy on the callee's stack
void concessionValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "  inside value fn : " << fee << '\n';
}
// 2. call by reference : parameter is an alias of the caller's variable
void concessionRef(long long& fee, int percent) { fee -= fee * percent / 100; }
// 3. call by pointer : parameter is a copy of the ADDRESS of the caller's variable
void concessionPtr(long long* fee, int percent)
{
    if (fee) *fee -= *fee * percent / 100;
}
// Design decision : return the new fee (pure function, composable)
long long concessionReturn(long long fee, int percent)
{
    return fee - fee * percent / 100;
}

struct Student { char name[64]; int marks[50]; long long fee; };

__attribute__((noinline)) long long readByValue(Student s)          { return s.fee + s.marks[0]; }
__attribute__((noinline)) long long readByRef(Student& s)           { return s.fee + s.marks[0]; }
__attribute__((noinline)) long long readByConstRef(const Student& s){ return s.fee +
s.marks[0]; }

int main()
{
    long long f1 = 45000, f2 = 45000, f3 = 45000;
    concessionValue(f1, 10);
    concessionRef(f2, 10);
    concessionPtr(&f3, 10);
    std::cout << "stored fee after value   : " << f1 << '\n';
    std::cout << "stored fee after ref     : " << f2 << '\n';
    std::cout << "stored fee after pointer : " << f3 << '\n';

    long long fee = 45000;      // merit 10%, sports 15%, single parent 25%
    long long inPlace = fee;
```

```
    concessionRef(inPlace, 10); concessionRef(inPlace, 15); concessionRef(inPlace, 25);
    long long returned = concessionReturn(concessionReturn(concessionReturn(fee, 10), 15),
    25);
    std::cout << "three concessions in place : " << inPlace  << '\n';
    std::cout << "three concessions returned : " << returned << " (original still " << fee <<
    ")\n";

    std::cout << "sizeof(long long) = " << sizeof(long long)
              << ", sizeof(Student) = " << sizeof(Student) << " bytes\n";

    Student s{"Asha", {90}, 45000};
    const int N = 20000000;
    long long sum = 0;
    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) { s.fee = i; sum += readByValue(s); }
    auto t1 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) { s.fee = i; sum += readByRef(s); }
    auto t2 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) { s.fee = i; sum += readByConstRef(s); }
    auto t3 = std::chrono::steady_clock::now();
    auto ms = [](auto a, auto b){ return std::chrono::duration<double, std::milli>(b - a).count();
    };
    std::cout << "20,000,000 calls  by value: " << ms(t0,t1) << " ms, by ref: " << ms(t1,t2)
              << " ms, by const ref: " << ms(t2,t3) << " ms (checksum " << sum << ")\n";
}
```

3. Prediction before running

Version	Predicted stored fee	Reason
Call by value	45000	Only the local copy of the fee is reduced
Call by reference	40500	45000 - 45000 × 10 / 100 = 45000 - 4500
Call by pointer	40500	The pointer holds the address of the caller's fee

4. Observed result

Output of q1_three_ways (compiled with -O0)

```
   inside value fn : 40500
stored fee after value   : 45000
stored fee after ref     : 40500
stored fee after pointer : 40500
three concessions in place : 25819
three concessions returned : 25819 (original still 45000)
sizeof(long long) = 8, sizeof(Student) = 272 bytes
20,000,000 calls  by value: 302.165 ms, by ref: 47.6475 ms, by const ref: 47.1366 ms
(checksum 600005370000000)
```

The prediction was correct. The value version prints 40500 **inside** the function (the local copy) but the stored fee remains 45000. The reference and pointer versions both change the stored fee to 40500.

5. Inquiry – where each version stores its parameter

Version	Where the parameter lives	Effect on the caller
By value	A new 8-byte object in the stack frame of the called function, holding a copy of the value	None. The copy is destroyed on return
By reference	No new object is visible to the programmer. The parameter is another name (alias) for the caller's variable; the compiler normally passes a hidden address	Caller's variable is changed directly
By pointer	A new 8-byte pointer variable in the callee's stack frame holding the address of the caller's variable	Caller's variable is changed through *fee; the pointer itself is only a copy

Only two versions change the caller's fee because only they give the function access to the **original object**. The by-value version gives access to a duplicate. A **const reference** (const Student&) should be used when the function only needs to read a large object: it avoids copying the object, it cannot modify the object, and it can also bind to temporaries. For small built-in types such as int or long long, passing by value is just as cheap and simpler.

6. Design decision – change in place or return the new fee?

Several concessions may apply to one student. **Decision: return the new fee** (long long concessionReturn(long long fee, int percent)) and let the caller assign it back where the record must really change, for example s.fee = concessionReturn(s.fee, 10);.

- **Composable:** concessionReturn(concessionReturn(concessionReturn(fee,10),15),25) is a single expression. With the in-place version three separate statements are needed.
- **Original fee is preserved:** the program above shows that after three concessions the in-place copy is 25819 while the original is still 45000. The hostel office needs the original fee for receipts and audit; an in-place update destroys it.
- **Easy to test:** a function with no side effects depends only on its arguments, so it can be tested with plain values.
- **Policy question made visible:** with in-place updates the order of concessions silently changes the result because of integer truncation. With returned values the order is visible in the expression.

The reference version is still a reasonable choice when exactly one function owns the update of a record and the original is never needed again.

7. Predict the Output

Given program

```
#include <iostream>
void change(int a, int& b, int* c)
{
	a += 10;
	b += 10; *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
	int a = 1, b = 2, c = 3;
	change(a, b, &c);
	std::cout << a << ' ' << b << ' ' << c << '\n';
	bigger(a, c) = 100;
	std::cout << a << ' ' << c << '\n';
	show(1);
	show(1, 2);
	show(1, 2, 3);
	std::cout << '\n';
}
```

My predicted output (written before running):

Prediction

```
1 12 13
1 100
16 13 6
```

Actual output

```
1 12 13
1 100
16 13 6
```

Difference between prediction and actual output: none. The only detail to notice is that the third line ends with a space after 6 because show() prints ' ' after every number.

Call	Which variables changed and why
change(a, b, &c)	a is passed by value, so the function adds 10 to its own copy and the caller's a stays 1. b is a reference, so the caller's b becomes 12. c is passed by pointer, so *c += 10 changes the caller's c to 13. Printed: 1 12 13.
bigger(a, c) = 100	bigger returns an int& to the larger variable. a is 1 and c is 13, so it returns a reference to c, and the assignment sets c to 100. a is unchanged. Printed: 1 100.
show(1)	y = 5 and z = 10 both come from default arguments: 1 + 5 + 10 = 16.

Call	Which variables changed and why
show(1, 2)	Only z = 10 is a default: 1 + 2 + 10 = 13.
show(1, 2, 3)	No default argument is used: 1 + 2 + 3 = 6.

8. Complete the Code

Completed program (TODO 1 replaced)

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;   // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
    // TODO 1 (completed): call by reference, integer arithmetic
    fee -= fee * percent / 100;
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Output

```
after call by value : 45000
after call by reference: 40500
```

The output matches the expected output (45000 and 40500). The program prints each message on its own line; the expected output in the assignment sheet appears on one line only because of the way it was typeset. **Why the call by value leaves the fee at 45000:** applyByValue receives a copy of hostelFee. The statement fee -= fee * percent / 100 reduces the copy to 40500, but the copy is destroyed when the function returns, and hostelFee in main was never touched.

Alternative Concept and Complexity Analysis

Alternative: function that returns the new fee

```
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
}
// in main:   hostelFee = applyConcession(hostelFee, 10);
```

The caller must now write the assignment itself. The function has no side effect, so it can be used inside expressions and for what-if calculations.

Passing method	Time	Bytes copied: long long	Bytes copied: Student (272 bytes)
By value	O(1)	8	272 (whole object copied)
By reference	O(1)	0 (alias; 8-byte address internally)	0 (alias)
By const reference	O(1)	0 (alias)	0 (alias, read-only)
Return new fee	O(1)	8 copied in, 8 returned	272 if the Student itself is passed and returned

The time is O(1) for every version because the arithmetic does not depend on the amount of data. The space copied per call is O(s), where s is the size of the object being passed. Our own measurement of 20,000,000 calls with the 272-byte Student object gave:

Passing method	Compiled with -O0	Compiled with -O2
By value	302.2 ms	271.0 ms
By reference	47.6 ms	12.7 ms
By const reference	47.1 ms	12.7 ms

Passing the large object by value is about 6 times slower at -O0 and about 21 times slower at -O2, because the copy of 272 bytes dominates the cost of the function. By reference and by const reference have the same cost; const reference is preferred for read-only use because the compiler rejects accidental modification.

Q2: Return by Reference and Its Dangers

1. Problem analysis

The marks register keeps marks in an array. highest() returns a reference to the highest mark so that the moderation of +2 can be applied directly to the array element. A second function that returns a reference to a local variable shows the danger of the same feature.

2. Implementation – returning a reference, a const reference and a copy

Source: q2_register.cpp

```
#include <iostream>
#include <vector>

int& highest(int m[], int n)                // return by reference
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}
const int& highestConst(const int m[], int n) // return by const reference
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}
int highestCopy(const int m[], int n)       // return by value (copy)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return m[idx];
}
void print(const int m[], int n)
{
    for (int i = 0; i < n; ++i) std::cout << m[i] << ' ';
    std::cout << '\n';
}
int main()
{
    int marks[5] = {62, 78, 91, 55, 84};
    std::cout << "original           : "; print(marks, 5);

    highest(marks, 5) += 2;                 // moderation applied directly
    std::cout << "after highest()+=2  : "; print(marks, 5);

    highest(marks, 5) = 95;                 // call on the LEFT of =
    std::cout << "after highest()=95  : "; print(marks, 5);

    int copy = highestCopy(marks, 5);
    copy += 5;                              // changes only the copy
    std::cout << "after copy += 5     : "; print(marks, 5);

    const int& cr = highestConst(marks, 5);    // read-only view
```

```
    // cr += 1;   // error: assignment of read-only reference 'cr'
    std::cout << "const reference read: " << cr << '\n';

    // ---- Changed condition : std::vector that grows ----
    std::vector<int> v = {62, 78, 91};
    int& r = v[2];
    const int* before = v.data();
    std::cout << "\ncapacity before growth: " << v.capacity() << '\n';
    for (int i = 0; i < 10; ++i) v.push_back(50 + i);   // forces reallocation
    const int* after = v.data();
    std::cout << "capacity after growth : " << v.capacity() << '\n';
    std::cout << "buffer moved          : " << std::boolalpha << (before != after) << '\n';
    std::cout << "r still refers to v[2]: " << (&r == &v[2]) << "  (false => r is dangling)\n";
}
```

Output

```
original           : 62 78 91 55 84
after highest()+=2  : 62 78 93 55 84
after highest()=95  : 62 78 95 55 84
after copy += 5     : 62 78 95 55 84
const reference read: 95

capacity before growth: 3
capacity after growth : 24
buffer moved          : true
r still refers to v[2]: false  (false => r is dangling)
```

3. What the output shows

Return type	Behaviour demonstrated
int& highest(...)	highest(marks,5) += 2 changed 91 to 93 inside the array. The call can also be used on the left of = (highest(marks,5) = 95) because it returns an lvalue.
const int& highestConst(...)	The caller sees the real element without copying it, but cannot change it. The statement cr += 1; is a compile-time error (assignment of read-only reference).
int highestCopy(...)	The caller receives a copy. copy += 5 left the array unchanged (95 stayed 95).

4. Returning a reference to a local variable

Source: q2_dangling.cpp

```
#include <iostream>
int& bad()
{
    int local = 42;
    return local;         // reference to a local variable
}
int main()
```

```
{
    int& r = bad();        // r refers to a destroyed object
    std::cout << r << '\n';// undefined behaviour
}
```

Compiler warning (g++ -std=c++17 -Wall -Wextra)

```
Q2/q2_dangling.cpp: In function 'int& bad()':
Q2/q2_dangling.cpp:5:12: warning: reference to local variable 'local' returned [-Wreturn-local-addr]
    5 |     return local;          // reference to a local variable
      |            ^~~~~
Q2/q2_dangling.cpp:4:9: note: declared here
    4 |     int local = 42;
      |         ^~~~~
```

Result of running the program

```
Segmentation fault   (exit code 139)
```

The compiler warns reference to local variable ‘local’ returned. The local variable is stored in the stack frame of bad(). When bad() returns, the frame is released and the object local is destroyed, so the returned reference **refers to an object that no longer exists** (a dangling reference). Reading it in main is **undefined behaviour**: the program may print 42, print garbage, crash or appear to work. With g++ 13 the program crashed with a segmentation fault; the exact behaviour changes with the compiler and optimisation level, which is why such code must never be relied on. A reference may only be returned if the object lives longer than the function call: an array element, a member of an object that still exists, a static variable or a reference parameter.

5. Changed condition – the register is a growing std::vector

A std::vector stores its elements in one contiguous block of memory. When push_back is called and the capacity is full, the vector allocates a larger block, copies or moves all elements to it and frees the old block. Any reference, pointer or iterator obtained before that now refers to the freed block and becomes **invalid (dangling)**. The demonstration in the output above shows capacity 3 growing to 24, the buffer moving, and &r == &v[2] becoming false: the reference r no longer refers to the vector’s element. A returned reference from highest() must therefore be used immediately and must not be stored across any operation that can grow the vector (push_back, insert, resize, reserve). Storing the **index** instead remains valid after growth.

Alternative Concept and Complexity Analysis

Source: q2_alternatives.cpp

```
#include <iostream>
#include <vector>
#include <chrono>

int  highestIndex(const std::vector<int>& m)
{ int idx = 0; for (int i = 1; i < (int)m.size(); ++i) if (m[i] > m[idx]) idx = i; return idx; }
int* highestPtr(std::vector<int>& m)
{ int* p = &m[0]; for (auto& x : m) if (x > *p) p = &x; return p; }
```

```
int& highestRef(std::vector<int>& m)
{ int* p = &m[0]; for (auto& x : m) if (x > *p) p = &x; return *p; }

int main()
{
    std::vector<int> marks = {62, 78, 91, 55, 84};
    int i = highestIndex(marks);  marks[i] += 2;       // index version
    *highestPtr(marks) += 2;                          // pointer version
    highestRef(marks)  += 2;                          // reference version
    std::cout << "highest after three +2 : " << marks[2] << " (expected 97)\n";

    int idx = highestIndex(marks);
    int* p  = highestPtr(marks);
    int& r  = highestRef(marks);
    const int* oldBuffer = marks.data();
    marks.reserve(1000);                              // vector grows -> reallocation
    std::cout << "buffer moved      : " << std::boolalpha << (oldBuffer != marks.data()) << '\n';
    std::cout << "index still valid : " << marks[idx] << '\n';
    std::cout << "pointer valid?    : " << (p == &marks[idx]) << '\n';
    std::cout << "reference valid?  : " << (&r == &marks[idx]) << '\n';

    std::vector<int> big(5000000);
    for (size_t k = 0; k < big.size(); ++k) big[k] = (int)((k * 2654435761u) % 1000003);
    long long s = 0;
    auto t0 = std::chrono::steady_clock::now();
    for (int k = 0; k < 20; ++k) s += highestIndex(big);
    auto t1 = std::chrono::steady_clock::now();
    for (int k = 0; k < 20; ++k) s += *highestPtr(big);
    auto t2 = std::chrono::steady_clock::now();
    for (int k = 0; k < 20; ++k) s += highestRef(big);
    auto t3 = std::chrono::steady_clock::now();
    auto ms = [](auto a, auto b){ return std::chrono::duration<double, std::milli>(b - a).count();
    };
    std::cout << "n=5,000,000 x20 : index " << ms(t0,t1) << " ms, pointer " << ms(t1,t2)
                 << " ms, reference " << ms(t2,t3) << " ms (" << s << ")\n";
}
```

Output (compiled with -O0)

```
highest after three +2 : 97 (expected 97)
buffer moved      : true
index still valid : 97
pointer valid?    : false
reference valid?  : false
n=5,000,000 x20 : index 510.641 ms, pointer 635.694 ms, reference 520.052 ms (51384900)
```

Return type	Calling code	Valid after vector grows?	Safety
int& (reference)	highestRef(v) += 2;	No	Short and clean but cannot be reseated; dangles after growth

Return type	Calling code	Valid after vector grows?	Safety
int* (pointer)	*highestPtr(v) += 2;	No	Can be null/dangling; caller must dereference
int (index)	v[highestIndex(v)] += 2;	Yes	Safest: index stays meaningful after reallocation

Time: all three scan the n marks once, so the time is O(n). **Space:** each uses a constant number of extra variables, so the space is O(1). Measured with 5,000,000 elements, 20 scans each: index 510.6 ms, pointer 635.7 ms, reference 520.1 ms (at -O0). The differences are small compared with the O(n) scan, so safety, not speed, should decide: the reference and pointer versions are fine for an immediate change, but the **index** is the safest value to return or store when the container can grow.

Q3: Default Arguments and Function Overloading

1. The compile error – an ambiguous call

The developer wrote three functions: charge(double), charge(double, bool) and charge(double, bool = false, bool = false).

Source: q3_ambiguous.cpp

```
#include <iostream>
double charge(double weight)              { return weight * 50; }
double charge(double weight, bool express) { return weight * 50 + (express ? 40 : 0); }
double charge(double weight, bool express = false, bool insured = false)
{ return weight * 50 + (express ? 40 : 0) + (insured ? 20 : 0); }
int main()
{
    std::cout << charge(2.5) << '\n';          // ambiguous: overload 1 or 3 ?
    std::cout << charge(2.5, true) << '\n';    // ambiguous: overload 2 or 3 ?
}
```

Compiler output (errors only are essential)

```
Q3/q3_ambiguous.cpp: In function 'int main()':
Q3/q3_ambiguous.cpp:8:24: error: call of overloaded 'charge(double)' is ambiguous
    8 |     std::cout << charge(2.5) << '\n';          // ambiguous: overload 1 or 3 ?
      |                  ~~~~~~^~~~~
Q3/q3_ambiguous.cpp:2:8: note: candidate: 'double charge(double)'
    2 | double charge(double weight)              { return weight * 50; }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool, bool)'
    4 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
Q3/q3_ambiguous.cpp:9:24: error: call of overloaded 'charge(double, bool)' is ambiguous
    9 |     std::cout << charge(2.5, true) << '\n';    // ambiguous: overload 2 or 3 ?
      |                  ~~~~~~^~~~~~~~~~~
Q3/q3_ambiguous.cpp:3:8: note: candidate: 'double charge(double, bool)'
    3 | double charge(double weight, bool express) { return weight * 50 + (express ? 40 : 0); }
      |        ^~~~~~
Q3/q3_ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool, bool)'
    4 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
```

The function definitions themselves compile, but the **calls** do not. The call charge(2.5) matches charge(double) exactly and also matches the third function exactly (the two bool parameters are filled by default arguments). Neither candidate is better, so the compiler reports an ambiguous call. In the same way charge(2.5, true) matches both the second and the third function exactly. A default argument does not make a function “less preferred”: during overload resolution a function whose missing arguments are supplied by defaults is just as good as one that needs no defaults. The first and second functions are therefore redundant and must be removed, because the third already covers one, two and three arguments.

2. Redesign

The design has **one function per kind of input**, each with default arguments for the optional flags:

- charge(int grams, bool express = false, bool insured = false) – weight in grams.
- charge(double kg, bool express = false, bool insured = false) – weight in kilograms.
- charge(const Parcel& p, bool express = false, bool insured = false) – a Parcel object (kg and a fragile flag).

No two functions differ only by default arguments, and the three first-parameter types (int, double, Parcel) are different, so a call is resolved by its first argument. Common calls such as charge(2.5) and charge(p, true) stay short.

Source: q3_charge.cpp

```
#include <iostream>
#include <iomanip>

struct Parcel { double kg; bool fragile; };

// weight in grams (int)
double charge(int grams, bool express = false, bool insured = false)
{
    std::cout << "[grams version]  ";
    return 50.0 * grams / 1000 + (express ? 40 : 0) + (insured ? 20 : 0);
}
// weight in kilograms (double)
double charge(double kg, bool express = false, bool insured = false)
{
    std::cout << "[kg version]     ";
    return 50.0 * kg + (express ? 40 : 0) + (insured ? 20 : 0);
}
// a Parcel object (fragile parcels pay a Rs 15 handling fee)
double charge(const Parcel& p, bool express = false, bool insured = false)
{
    std::cout << "[Parcel version] ";
    return 50.0 * p.kg + (p.fragile ? 15 : 0) + (express ? 40 : 0) + (insured ? 20 : 0);
}

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    Parcel p{3.0, true};
    double r;
    r = charge(2);              std::cout << r << "  <- charge(2)\n";
    r = charge(2.0f);           std::cout << r << "  <- charge(2.0f)\n";
    r = charge(2.5);            std::cout << r << "  <- charge(2.5)\n";
    r = charge(2500, true);     std::cout << r << "  <- charge(2500, true)\n";
    r = charge(p);              std::cout << r << "  <- charge(p)\n";
    r = charge(2.5, true, true); std::cout << r << "  <- charge(2.5, true, true)\n";
}
```

3. Prediction before execution

Call	Predicted function	Reason
charge(2)	grams (int)	2 is an int: exact match with the int overload (so it means 2 grams)
charge(2.0f)	kg (double)	float to double is a promotion; float to int is a conversion; promotion is better
charge(2.5)	kg (double)	double literal: exact match
charge(2500, true)	grams (int)	int and bool: exact match on both parameters
charge(p)	Parcel	Parcel object: only the Parcel overload can accept it
charge(2.5, true, true)	kg (double)	double, bool, bool: exact match

4. Observed result

Output

```
[grams version]  0.10  <- charge(2)
[kg version]     100.00  <- charge(2.0f)
[kg version]     125.00  <- charge(2.5)
[grams version]  165.00  <- charge(2500, true)
[Parcel version] 165.00  <- charge(p)
[kg version]     185.00  <- charge(2.5, true, true)
```

All six predictions were correct. Note the trap in charge(2): the literal 2 is an int, so it is 2 **grams** and costs only Rs 0.10, not 2 kg. This is a design weakness of using int for grams and double for kilograms; a caller who means 2 kg must write charge(2.0). A call such as charge(2L) (long) is **ambiguous** because long to int and long to double are both conversions of equal rank; this was confirmed with g++ 13 ("call of overloaded charge(long int) is ambiguous").

5. Inquiry – how the compiler chooses, and why defaults go from the right

For every call the compiler first builds the set of functions with that name that can accept the arguments (the arguments are matched to parameters, and default arguments fill missing ones). It then ranks each argument conversion and selects the function that is better for at least one argument and not worse for any:

- **Exact match** – same type (or array-to-pointer, adding const); best.
- **Promotion** – for example float to double, char or short to int.
- **Conversion** – for example int to double, double to int, long to int, derived pointer to base pointer.
- **User-defined conversion** – a converting constructor or conversion operator; worst.

If no single function is best, the call is ambiguous and is a compile-time error. The return type is **not** used.

Default arguments must be given from the right because arguments are matched to parameters from left to right. If a call supplies fewer arguments than parameters, the missing ones are always the rightmost ones. A declaration such as f(int a = 1, int b) would leave no way to call f with only b: the single argument given would always be taken as a. Therefore once a parameter has a default, every parameter to its right must also have one. A default may be given only once for a parameter in a scope, normally in the declaration.

Alternative Concept and Complexity Analysis

Source: q3_alternative.cpp

```
#include <iostream>
#include <iomanip>
// Alternative 1 : ONE function taking a Parcel with optional fields
struct Parcel2
{
    double kg      = 0;
    bool   fragile = false;
    bool   express = false;
    bool   insured = false;
};
double chargeP(const Parcel2& p)
{ return 50.0 * p.kg + (p.fragile ? 15 : 0) + (p.express ? 40 : 0) + (p.insured ? 20 : 0); }

// Alternative 2 : function template (one body for any numeric weight type)
template <typename W>
double chargeT(W weight, bool express = false)
{ return 50.0 * static_cast<double>(weight) + (express ? 40 : 0); }

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    Parcel2 p; p.kg = 2.5; p.insured = true;
    std::cout << chargeP(p) << '\n';
    std::cout << chargeT(2) << ' ' << chargeT(2.5) << ' ' << chargeT(2.0f, true) << '\n';
}
```

Output

```
145.00
100.00 125.00 140.00
```

Alternative 1 – a single Parcel structure with optional fields. All options are members of one structure with default member initialisers, and there is exactly one chargeP(const Parcel2&) function. **Alternative 2 – a function template** chargeT<W>(W weight, bool) that works for int, float or double weight with one body.

	Overloads + defaults	**Parcel structure**	**Function template**
Overload resolution time	Compile time only, O(1) at run time	None (one function)	Template deduction at compile time

	Overloads + defaults	**Parcel structure**	**Function template**
Space copied per call	Arguments only (8 + 1 + 1 bytes)	One struct passed by const reference: 0 copied, struct about 16 bytes	Same as the argument types
Adding a new option (for example “fragile”)	Edit every overload and every call may become ambiguous	Add one field with a default; old calls remain valid	Add a parameter; all instantiations change
Risk	Silent wrong overload, e.g. charge(2) means grams	Caller must name fields; no accidental overload	int weight is silently treated as kg (charge(2) = 2 kg)

Overload resolution has **no run-time cost** in any of the designs: it is finished before the program runs, so the time per call is O(1) and the same function body is executed. The structure approach scales best when options grow because a new option is one new field with a default value, and old calls keep working. The overload approach gives the most natural calls for the common cases but is the most fragile when new options are added.

Q4: Specifying a Class with an Array Inside It

1. Class specification

Each SavingsAccount keeps its account number, balance, an array of 10 transactions and a count. Member functions are defined **inside** the class (deposit, record, getBalance, which become implicitly inline) and **outside** the class using the scope resolution operator (SavingsAccount::withdraw, SavingsAccount::miniStatement).

Source: q4_account.cpp

```
#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };   // 'D' deposit, 'W' withdrawal

class SavingsAccount {
    static const int MAX = 10;       // change to 50 for the 50-transaction requirement
    int        accNo;
    double     balance;
    Transaction last[MAX];           // private: only the class keeps it consistent
    int        count;                // number of valid transactions (0..MAX)

    void record(char t, double amt)   // defined inside the class (implicitly inline)
    {
        if (count == MAX) {          // 11th transaction: shift left, discard the oldest
            for (int i = 1; i < MAX; ++i) last[i - 1] = last[i];
            --count;
        }
        last[count].type = t;
        last[count].amount = amt;
        ++count;
    }
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), count(0) {}
    void   deposit(double amt) { balance += amt; record('D', amt); }   // inside
    bool   withdraw(double amt);                                       // outside
    void   miniStatement() const;                                      // outside
    double getBalance() const { return balance; }
};

bool SavingsAccount::withdraw(double amt)
{
    if (amt > balance) return false;          // insufficient funds: nothing recorded
    balance -= amt;
    record('W', amt);
    return true;
}
void SavingsAccount::miniStatement() const
{
    std::cout << "Mini-statement for account " << accNo << '\n';
    for (int i = 0; i < count; ++i)
        std::cout << std::setw(3) << i + 1 << ". "
                  << (last[i].type == 'D' ? "Deposit    " : "Withdrawal")
```

```
                        << std::setw(9) << last[i].amount << '\n';
    std::cout << "Balance: " << balance << '\n';
}

int main()
{
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 8; ++i) acc.deposit(i * 100);   // 100 .. 800
    acc.withdraw(250);
    acc.withdraw(300);
    acc.deposit(1000);
    if (!acc.withdraw(100000)) std::cout << "Withdrawal of 100000 refused\n";
    acc.withdraw(50);                                    // 12th accepted transaction
    acc.miniStatement();                                  // only the last 10 are shown
}
```

Output

```
Withdrawal of 100000 refused
Mini-statement for account 1001
   1. Deposit        300
   2. Deposit        400
   3. Deposit        500
   4. Deposit        600
   5. Deposit        700
   6. Deposit        800
   7. Withdrawal     250
   8. Withdrawal     300
   9. Deposit       1000
  10. Withdrawal      50
Balance: 9000
```

Twelve transactions were accepted (8 deposits, 2 withdrawals, 1 deposit, 1 withdrawal; the withdrawal of 100000 was refused and not recorded). Only the last 10 are printed; the deposits of 100 and 200 were discarded. The balance is 5000 + 3600 − 250 − 300 + 1000 − 50 = 9000, which matches.

2. Inquiry

Why the array must be private. The array and the count must always agree: count is the number of valid entries and the order of the entries is the order of the transactions. If the array were public, any code could write to last[7], change the count, reorder or delete entries, and the mini-statement would no longer be a trustworthy record. Keeping the array private means only deposit, withdraw and record can change it, and all rules (such as discarding the oldest entry) are enforced in one place. The internal representation can also be changed later (for example to a circular buffer) without changing any code that uses the class.

The 11th transaction. Two ways of discarding the oldest entry were written:

Design	What happens on the 11th transaction	Cost
Shift the array	Every element is copied one place to the left (last[i-1] = last[i], K-1 copies), then the new transaction is written at the end	O(K) per transaction (9 copies for K = 10); entries stay in order, so printing is a simple loop
Circular index	The new transaction overwrites the oldest slot and the start index advances by one (modulo K); nothing is copied	O(1) per transaction; printing needs (start + i) % K

Circular-index version (same public interface)

```
#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };

class SavingsAccount {
    static const int MAX = 10;
    int        accNo;
    double     balance;
    Transaction last[MAX];
    int        start;          // index of the oldest transaction
    int        count;
    void record(char t, double amt)
    {
        int pos = (start + count) % MAX;     // next free slot (or the oldest when full)
        last[pos].type = t; last[pos].amount = amt;
        if (count == MAX) start = (start + 1) % MAX;   // overwrite oldest: O(1)
        else              ++count;
    }
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), start(0), count(0)
    {}
    void deposit(double amt) { balance += amt; record('D', amt); }
    bool withdraw(double amt);
    void miniStatement() const;
};
bool SavingsAccount::withdraw(double amt)
{
    if (amt > balance) return false;
    balance -= amt; record('W', amt); return true;
}
void SavingsAccount::miniStatement() const
{
    std::cout << "Mini-statement for account " << accNo << '\n';
    for (int i = 0; i < count; ++i) {
        const Transaction& t = last[(start + i) % MAX];
        std::cout << std::setw(3) << i + 1 << ". "
                  << (t.type == 'D' ? "Deposit   " : "Withdrawal")
```

```
                    << std::setw(9) << t.amount << '\n';
        }
        std::cout << "Balance: " << balance << '\n';
}
int main()
{
        SavingsAccount acc(1001, 5000);
        for (int i = 1; i <= 8; ++i) acc.deposit(i * 100);
        acc.withdraw(250); acc.withdraw(300); acc.deposit(1000);
        if (!acc.withdraw(100000)) std::cout << "Withdrawal of 100000 refused\n";
        acc.withdraw(50);
        acc.miniStatement();
}
```

Output of the circular version

```
Withdrawal of 100000 refused
Mini-statement for account 1001
   1. Deposit         300
   2. Deposit         400
   3. Deposit         500
   4. Deposit         600
   5. Deposit         700
   6. Deposit         800
   7. Withdrawal      250
   8. Withdrawal      300
   9. Deposit        1000
  10. Withdrawal       50
Balance: 9000
```

The mini-statement is identical in both versions, so the two designs are interchangeable from the outside.

3. Design decision – the last 50 transactions

Both designs change least in the same way: the capacity is the single constant static const int MAX, so changing MAX = 10 to MAX = 50 changes the array size, the discard test and the loops automatically; no other line is edited and the public interface (deposit, withdraw, miniStatement) is untouched. In terms of **behaviour**, however, the shifting design gets more expensive (49 copies per transaction instead of 9), while the circular design stays at one write. **Defended choice:** keep the class, change the constant to 50, and use the circular index version, because it needs no further change when the requirement changes again (for 500 or 5000) and its cost does not grow with K. If the capacity must differ per account, it can become a constructor argument with a std::vector or a template parameter; this is more change but still hidden inside the class.

Alternative Concept and Complexity Analysis

std::deque version (push_back and pop_front)

```
#include <iostream>
#include <deque>
#include <iomanip>
```

```
struct Transaction { char type; double amount; };
class SavingsAccount {
	static const std::size_t MAX = 10;
	int accNo; double balance; std::deque<Transaction> last;
	void record(char t, double a)
	{
		if (last.size() == MAX) last.pop_front();   // O(1)
		last.push_back({t, a});                     // O(1)
	}
public:
	SavingsAccount(int no, double op) : accNo(no), balance(op) {}
	void deposit(double a) { balance += a; record('D', a); }
	bool withdraw(double a) { if (a > balance) return false; balance -= a; record('W', a); return
	true; }
	void miniStatement() const
	{
		std::cout << "Mini-statement for account " << accNo << '\n';
		int i = 1;
		for (const auto& t : last)
			std::cout << std::setw(3) << i++ << ". " << (t.type == 'D' ? "Deposit    " :
			"Withdrawal")
					<< std::setw(9) << t.amount << '\n';
		std::cout << "Balance: " << balance << '\n';
	}
};
int main()
{
	SavingsAccount acc(1001, 5000);
	for (int i = 1; i <= 8; ++i) acc.deposit(i * 100);
	acc.withdraw(250); acc.withdraw(300); acc.deposit(1000);
	acc.withdraw(50);
	acc.miniStatement();
}
```

Output of the deque version (same statement)

```
Mini-statement for account 1001
  1. Deposit        300
  2. Deposit        400
  3. Deposit        500
  4. Deposit        600
  5. Deposit        700
  6. Deposit        800
  7. Withdrawal     250
  8. Withdrawal     300
  9. Deposit       1000
 10. Withdrawal      50
Balance: 9000
```

Design	Time to add a transaction	Space	Remarks
Fixed array, shift left	O(K) when full (K-1 copies)	O(K): 10 × 16 bytes = 160 bytes + count	Simplest; contiguous; order = array order
Circular buffer (start index)	O(1)	O(K): same array + one int (start)	No copying; slightly more complex indexing
std::deque (push_back, pop_front)	O(1) amortised	O(K) plus block and bookkeeping overhead (heap allocation)	Grows dynamically; no fixed bound unless enforced by the class

For K = 10 all three are fast enough for an ATM. The array-based designs keep all data inside the object (no heap allocation, good cache behaviour); the deque adds heap allocation and is more than needed when the capacity is fixed. The time measurements for different K are reported in Q7.

Q5: Static Members and Arrays of Objects

1. From the principle to a class design

The principle is: data that belongs to the whole class, not to any one object, should exist only once however many objects are created. For the college bus this gives the following design.

Item	Design decision
Which members are static	issued (number of seats issued so far) must be static, because there is one count for the whole bus, not one per ticket. CAPACITY is a static const, since the capacity is the same for all tickets. count() is a static member function so the transport office can ask “how many seats are issued?” without any ticket object: BusTicket::count().
Which members are not static	seat and name – each ticket has its own seat number (0 means waiting list) and passenger name.
Where a static data member is defined	Declaring static int issued; inside the class only declares it. It must be **defined once, outside the class, in exactly one source file**: int BusTicket::issued = 0;. Without this definition the linker reports “undefined reference”. (Integral static const members with an in-class initialiser, like CAPACITY, can be used by value without a separate definition.)
Why a static member function cannot use this or non-static members	A static member function is not called on an object, so it has **no this pointer**. Non-static members (seat, name) belong to a particular object; with no object there is nothing to say whose seat is meant. It can use only static members and its parameters.
How an array of objects is created	BusTicket trip[4] = { BusTicket("Asha"), ... }; Each element is constructed in order, element 0 first. Because the constructor is explicit BusTicket(const std::string&) and there is no default constructor, every element must be initialised explicitly; BusTicket trip[4]; would not compile.
What happens to the count in a loop or function	The static count is not tied to the lifetime of any object. Every constructor call that issues a seat increases it and it stays increased after the objects are destroyed, since no destructor reduces it. Demonstrated in 5.5.

2. Predict the Output

Given program

```
#include <iostream>
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main(){
```

```
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
              << ' ' << Token::issued() << '\n';
}
```

My predicted output (written before running):

Prediction

```
101 103 104 4
```

Actual output

```
101 103 104 4
```

Difference between prediction and actual output: none.

- int Token::next = 100; is the definition of the static member. It is initialised once, before main starts, so the shared counter **starts at 100**, not 0.
- The array counter[3] runs the constructor three times: ++next gives 101, 102, 103, so counter[0].get() is 101 and counter[2].get() is 103.
- extra runs the constructor a fourth time and gets 104.
- In total the **constructor ran 4 times**. Token::issued() returns next − 100 = 104 − 100 = 4, and it can be called without an object because it is static.

3. Complete the Code

Completed program (TODO 1 and TODO 2 replaced)

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;              // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1 (completed)
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }
    static int count()
    {
        // TODO 2 (completed)
        return issued;
    }
    void print() const
    {
        std::cout << name << ": "
```

```
                << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                << '\n';
    }
};
int BusTicket::issued = 0;          // definition of the static data member
int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Output

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

The output is identical to the expected output. The array has four tickets and the capacity is three, so Dev receives seat 0 and is printed as “Waiting”. The test with an array larger than the capacity confirms the seat numbers 1, 2, 3 and the waiting list.

4. Test: tickets created inside a function called twice

Source: q5_function.cpp

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat; std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    { if (issued < CAPACITY) seat = ++issued; }
    ~BusTicket() { std::cout << "  (ticket of " << name << " destroyed)\n"; }
    static int count() { return issued; }
    void print() const
    { std::cout << name << ": " << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
    << '\n'; }
};
int BusTicket::issued = 0;

void book(const std::string& a, const std::string& b)
{
    BusTicket t1(a), t2(b);          // automatic objects
    t1.print(); t2.print();
}                                    // both destroyed here, count NOT reduced
int main()
{
    book("Asha", "Bala");
```

```
    std::cout << "Issued after call 1: " << BusTicket::count() << '\n';
    book("Chitra", "Dev");
    std::cout << "Issued after call 2: " << BusTicket::count() << '\n';
}
```

Output

```
Asha: Seat 1
Bala: Seat 2
  (ticket of Bala destroyed)
  (ticket of Asha destroyed)
Issued after call 1: 2
Chitra: Seat 3
Dev: Waiting
  (ticket of Dev destroyed)
  (ticket of Chitra destroyed)
Issued after call 2: 3
```

Each call creates two ticket objects as local variables and they are destroyed when the function ends (the destructor messages show this). The count, however, is **not reduced**: it stays at 2 after the first call, so the second call gives Chitra seat 3 and Dev must wait. The static member lives for the whole program, so it records tickets issued, not tickets currently alive. If the count should follow the number of live objects, a destructor would have to decrease it; for seat numbers the current behaviour is the correct one because a seat, once issued, is not given again.

Alternative Concept and Complexity Analysis

Alternative: a separate TicketOffice object passed to every BusTicket

```
#include <iostream>
#include <string>
class TicketOffice {                    // one office per bus: two buses can run together
    int capacity, issued;
public:
    explicit TicketOffice(int cap) : capacity(cap), issued(0) {}
    int nextSeat() { return issued < capacity ? ++issued : 0; }
    int count() const { return issued; }
};
class BusTicket {
    int seat; std::string name;
public:
    BusTicket(const std::string& n, TicketOffice& office) : seat(office.nextSeat()), name(n) {}
    void print() const
    { std::cout << name << ": " << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
    << '\n'; }
};
int main()
{
    TicketOffice bus1(2), bus2(2);
    BusTicket a("Asha", bus1), b("Bala", bus1), c("Chitra", bus1);
    BusTicket d("Dev", bus2), e("Esha", bus2);
    a.print(); b.print(); c.print(); d.print(); e.print();
    std::cout << "Bus1 issued: " << bus1.count() << ", Bus2 issued: " << bus2.count() << '\n';
```

```
}
```

Output (two buses running at once)

```
Asha: Seat 1
Bala: Seat 2
Chitra: Waiting
Dev: Seat 1
Esha: Seat 2
Bus1 issued: 2, Bus2 issued: 2
```

	Static data member	**Global variable**	**TicketOffice object**
Time per ticket	O(1)	O(1)	O(1)
Space	One counter	One counter	One counter per office (one per bus)
Encapsulation	Private to the class; only BusTicket can change it	Any code can change it; no protection	Private inside TicketOffice; changed only via nextSeat()
Two buses at once	No: one shared counter and one capacity for all tickets	No	Yes: bus1 and bus2 are independent (seen in the output)
Cost	None extra; simplest call: BusTicket("Asha")	Name clashes, hard to test	An office must be passed to every constructor

The static design is best for exactly one bus: no extra object is needed and the data is protected. A global variable gives the same time and space but no encapsulation. The TicketOffice design uses slightly more code but is the only one that supports two buses with different capacities, as shown by the output (bus 1 with capacity 2 put Chitra on the waiting list while bus 2 still issued seats 1 and 2).

Q6: Friend Functions and Inline Functions

1. Design of the comparison

- **Why a friend function is needed when a function works on two objects equally.** A member function is called on one object (a.minutesTo(b)), so one object is "the" object and the other is only an argument; the call is not symmetric. A non-member function treats both equally, minutesBetween(a, b), but a plain non-member cannot read the private h and m. Declaring it a **friend** gives these functions, and only these, access to the private members of both objects without making the data public.
- **How a friend differs from a member.** A member function is called with the dot operator on an object, has a hidden this pointer, is in the scope of the class and can use the private members of its own object. A friend function is **not a member**: it has no this pointer, is called as an ordinary function f(a, b), is not in the class scope, and must use an object name (a.h) to reach private members. Friendship is granted by the class, it is not mutual, not inherited and not transitive.
- **Should the small functions be inline?** Yes, as a request. They contain a few arithmetic operations and are called millions of times, so the cost of the call (pushing arguments, jump, return) can be a large share of the total. inline asks the compiler to substitute the body at the call site, avoiding the call; it is only a request, and the compiler is free to ignore it (and does at -O0) or to inline a function that is not marked inline. It also allows the definition to appear in a header without a multiple-definition error.
- **What is lost if h and m are made public.** Any code could set minutes to 75 or hours to −3, so the class could no longer guarantee a valid time; no validation could be added later without breaking every user of the class, and the internal representation (for example "minutes since midnight") could never be changed. Friends keep the data private and widen access only to two named functions.

2. Complete the Code

Completed program (TODO 1 and TODO 2 replaced; extra tests added in main)

```
#include <iostream>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1 (completed)
    int ta = a.h * 60 + a.m;
    int tb = b.h * 60 + b.m;
    return ta > tb ? ta - tb : tb - ta;
}
bool earlier(const Time& a, const Time& b)
{
    // TODO 2 (completed)
    return a.h * 60 + a.m < b.h * 60 + b.m;
```

```
}
int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';

    // correctness checks
    Time a(11, 59), b(12, 1), c(12, 0), d(12, 0);
    std::cout << "11:59 vs 12:01 -> " << minutesBetween(a, b) << ' ' << earlier(a, b) << ' ' <<
    earlier(b, a) << '\n';
    std::cout << "12:00 vs 12:00 -> " << minutesBetween(c, d) << ' ' << earlier(c, d) << ' ' <<
    earlier(d, c) << '\n';
    Time e(0, 0), f(23, 59);
    std::cout << "00:00 vs 23:59 -> " << minutesBetween(e, f) << ' ' << earlier(e, f) << '\n';
}
```

Output

```
205 false
11:59 vs 12:01 -> 2 true false
12:00 vs 12:00 -> 0 false false
00:00 vs 23:59 -> 1439 true
```

3. Correctness check (verified by hand)

Case	Hand calculation	Program result
9:45 and 13:10	585 and 790 minutes; difference 205; 13:10 is not earlier than 9:45	205 false
11:59 and 12:01 (either side of noon)	719 and 721 minutes; difference 2; 11:59 is earlier than 12:01	2, earlier(a,b) = true, earlier(b,a) = false
12:00 and 12:00 (same time twice)	720 and 720; difference 0; neither is earlier	0, false, false
00:00 and 23:59	0 and 1439 minutes; difference 1439	1439 true

All results agree with the hand calculation. Note that the function assumes both times are on the **same day**; a journey that crosses midnight would need a day number.

4. Growth investigation – 10,000,000 calls with and without inline

Benchmark program (friend inline, friend noinline, member function, public getters)

```
#include <iostream>
#include <chrono>
#include <vector>
#include <algorithm>

class Time {
```

```
    int h, m;
public:
    Time(int hh = 0, int mm = 0) : h(hh), m(mm) {}
    // Design 1 : friend functions (inline keyword, defined in class)
    friend inline int minutesBetweenInline(const Time& a, const Time& b)
    { int x = a.h*60+a.m, y = b.h*60+b.m; return x > y ? x - y : y - x; }
    // Design 1b : same friend, forbidden from inlining
    friend __attribute__((noinline)) int minutesBetweenNoInline(const Time& a, const Time&
    b)
    { int x = a.h*60+a.m, y = b.h*60+b.m; return x > y ? x - y : y - x; }
    // Design 2 : member function
    int minutesTo(const Time& b) const
    { int x = h*60+m, y = b.h*60+b.m; return x > y ? x - y : y - x; }
    // Design 3 : public getters
    int getH() const { return h; }
    int getM() const { return m; }
};
int viaGetters(const Time& a, const Time& b)
{ int x = a.getH()*60+a.getM(), y = b.getH()*60+b.getM(); return x > y ? x - y : y - x; }

template <typename F>
double run(const std::vector<Time>& t, F f, long long& sink)
{
    double best = 1e18;
    for (int rep = 0; rep < 5; ++rep) {
        long long s = 0;
        auto t0 = std::chrono::steady_clock::now();
        for (int i = 0; i < 10000000; ++i) s += f(t[i & 1023], t[(i * 7 + 3) & 1023]);
        auto t1 = std::chrono::steady_clock::now();
        best = std::min(best, std::chrono::duration<double, std::milli>(t1 - t0).count());
        sink += s;
    }
    return best;
}
int main()
{
    std::vector<Time> t;
    for (int i = 0; i < 1024; ++i) t.emplace_back((i * 13) % 24, (i * 29) % 60);
    long long sink = 0;
    std::cout << "10,000,000 calls (best of 5), ms:\n";
    std::cout << "friend, inline      : " << run(t, [](const Time& a, const Time& b){ return
    minutesBetweenInline(a,b); }, sink) << '\n';
    std::cout << "friend, noinline    : " << run(t, [](const Time& a, const Time& b){ return
    minutesBetweenNoInline(a,b); }, sink) << '\n';
    std::cout << "member minutesTo    : " << run(t, [](const Time& a, const Time& b){ return
    a.minutesTo(b); }, sink) << '\n';
    std::cout << "public getters      : " << run(t, [](const Time& a, const Time& b){ return
    viaGetters(a,b); }, sink) << '\n';
    std::cout << "checksum " << sink << '\n';
}
```

The benchmark calls each design 10,000,000 times with varying data and reports the best of 5 runs. “Without inline” is forced with __attribute__((noinline)). Programs were compiled with g++ -std=c++17 -Wall -Wextra -O0 and -O2.

Result at -O0

```
10,000,000 calls (best of 5), ms:
friend, inline      : 78.8616
friend, noinline    : 109.825
member minutesTo    : 75.8873
public getters      : 122.534
checksum 102632831840
```

Result at -O2

```
10,000,000 calls (best of 5), ms:
friend, inline      : 12.8615
friend, noinline    : 14.8003
member minutesTo    : 13.1847
public getters      : 15.9699
checksum 102632831840
```

Design (10,000,000 calls)	**-O0 (ms)**	**-O2 (ms)**	**O0 / O2**
Friend, inline	78.9	12.9	6.1
Friend, noinline	109.8	14.8	7.4
Member minutesTo	75.9	13.2	5.8
Public getters	122.5	16.0	7.7

Explanation. (1) The largest effect is the optimisation level: -O2 is about 6 to 8 times faster than -O0 for every design because the optimiser keeps values in registers, removes the call overhead and simplifies the loop. (2) At -O0 the compiler does **not** inline, even for functions marked inline; the keyword is only a request. The difference between “friend inline” and “friend noinline” at -O0 therefore does not come from real inlining, and should be treated as measurement and code-layout variation; the getters version is slower because it makes four additional unoptimised calls per comparison. (3) At -O2 the inline friend and the member function (13 ms) are the fastest; the non-inlined friend and the getters version are only about 2 to 3 ms slower in 10,000,000 calls, i.e. 0.2 to 0.3 ns per call. The gain is small because the function body is tiny and the loop is dominated by the memory loads; the advantage of inlining is that the compiler can optimise the body together with the surrounding loop. (4) The three designs have the same cost per call, O(1), once the compiler has inlined them.

5. Deployment decision

Decision: keep the two friend functions (or equivalently make earlier an operator<), define them inline in the header, and do not add getters. The reasons are:

- The measurements show no significant run-time difference between friend and member designs once optimisation is on, so performance does not decide the matter.

- The comparison is symmetric: minutesBetween(a, b) and earlier(a, b) treat both times equally, which is the clearest reading for two objects of the same type.
- Encapsulation is best with friends: only two named functions can see h and m. Public getters expose h and m to **all** code, forever, and add the slowest design at -O0.
- A member function (a.minutesTo(b)) would be equally good for encapsulation, and is the better choice if the operation is naturally "from this time to another". The friend design is preferred for the symmetric comparison.

Alternative Concept and Complexity Analysis

Alternative designs (pseudocode)

```
// Member functions
class Time { int h, m;
public:
    int  minutesTo(const Time& b) const { int x=h*60+m, y=b.h*60+b.m; return x>y ? x-y : y-x; }
    bool isEarlier(const Time& b) const { return h*60+m < b.h*60+b.m; } };
// call:  a.minutesTo(b)    a.isEarlier(b)

// Public getters
class Time { int h, m; public: int getH() const {return h;} int getM() const {return m;} };
int minutesBetween(const Time& a, const Time& b)
{ int x=a.getH()*60+a.getM(), y=b.getH()*60+b.getM(); return x>y ? x-y : y-x; }
```

	Friend functions	**Member functions**	**Public getters**
Time per call	O(1)	O(1)	O(1)
Measured at -O2 (10M calls)	12.9 ms (inline)	13.2 ms	16.0 ms
Measured at -O0 (10M calls)	78.9 ms (inline)	75.9 ms	122.5 ms
Space	No extra data	No extra data	No extra data
Who can see h and m	Only the two friends	Only members of Time	Every part of the program, via getters
Symmetry of the call	Yes: f(a, b)	No: a.f(b)	Yes: f(a, b)

All three are O(1) per call and use no extra space. Friends and members give the best encapsulation. Getters give the least encapsulation and are slowest when the compiler does not inline.

Q7: Diagnose, Redesign and Validate

Program selected for redesign: the SavingsAccount mini-statement from Q4 (fixed array that is shifted to the left when it is full). Evidence from Q4 shows that the cost of every transaction depends on the number of transactions kept, which becomes a limitation when the requirement changes from 10 to 50 transactions or more.

A. Diagnose the Limitation (2 marks)

In the Q4 design record() runs the loop for (i = 1; i < MAX; ++i) last[i-1] = last[i]; on **every** transaction once the array is full, so each new transaction copies MAX − 1 elements of 16 bytes. The evidence generated by the benchmark of the original design (2,000,000 transactions, compiled with -O2):

K (kept)	Element copies per transaction (measured)	Time, original (ms)	Time per transaction (ns)
10	9.0	18.0	9.0
50	49.0	30.4	15.2
500	499.0	125.0	62.5
5000 (200,000 transactions)	4957	323.1	1,615

The time per transaction grows almost in proportion to K. The characteristic of the original design that causes this is that it keeps the transactions **in physical array order with the oldest at index 0**, so removing the oldest entry forces every other entry to move. The change from 10 to 50 transactions in Q4.3 therefore makes the original design about 1.7 times slower per transaction (O2), and 3.0 times slower at -O0 (49.8 ms for K = 10 to 149.6 ms for K = 50).

B. Define the Design Objective (2 marks)

Objective: make the cost of recording a transaction independent of the number of transactions kept, so that:

- zero elements are copied per transaction (moves per transaction = 0) for every K;
- at -O2 the redesign is at least 5 times faster than the original for K = 10 and at least 20 times faster for K = 500;
- the time per transaction at K = 500 is less than 2 times the time at K = 10;
- the printed mini-statement is identical to the original for the same sequence of transactions (correctness must not change).

C. Design the Revised Solution (4 marks)

What changes: the private data gets one extra integer, start, the index of the oldest transaction. The array is treated as a **circular buffer**. A new transaction is written at position (start + count) mod K; if the buffer is full this is exactly the oldest slot, which is overwritten, and start advances by one. Nothing is shifted. miniStatement() prints the entries beginning at start and wrapping

round. The public interface (deposit, withdraw, miniStatement) is unchanged, so no calling code is edited; this is possible because the array is private.

Class diagram (text form)

```
+-------------------------------------------+
|            SavingsAccount                 |
+-------------------------------------------+
| - accNo   : int                           |
| - balance : double                        |
| - last[K] : Transaction   (circular buffer) |
| - start   : int          <-- NEW          |
| - count   : int                           |
+-------------------------------------------+
| + deposit(amt)                            |
| + withdraw(amt) : bool                    |
| + miniStatement() const   (reads from start)|
| - record(type, amt)      <-- CHANGED      |
+-------------------------------------------+
```

Pseudocode of record()

```
record(type, amt):
    pos <- start + count
    if pos >= K then pos <- pos - K        // wrap round
    last[pos] <- (type, amt)
    if count == K then                     // full: oldest slot was overwritten
        start <- start + 1;  if start == K then start <- 0
    else
        count <- count + 1

miniStatement():
    for i from 0 to count - 1:
        p <- start + i;  if p >= K then p <- p - K
        print last[p]
```

- **C++ features used:** a private array member, the scope of a class (the representation is hidden), a static const int / template parameter for the capacity, and index arithmetic with a wrap-around test (a subtraction instead of the more expensive % in the hot path).
- **Extra memory:** one int (4 bytes) per account. The array size does not change: K × 16 bytes. The space stays O(K).
- **New trade-offs:** (1) slightly more complex code and the logical order is no longer the physical order, so the array cannot be copied or printed in one simple pass; (2) an off-by-one in start/count is a new place for bugs, so a test comparing both designs is essential; (3) for tiny K the gain is small in absolute terms.

D. Predict Before Implementing (1 mark)

Prediction (derived from the design analysis): the original design copies K − 1 elements per transaction, the redesign copies none. I therefore expect the **time per transaction** of the original to grow roughly linearly with K and that of the redesign to stay almost constant. Expected speed-up of the redesign: at least 3 times for K = 10, about 10 times for K = 50, and more than

50 times for K = 500. The measure is the total time for 2,000,000 transactions, together with the number of element moves per transaction (9, 49, 499 for the original and 0 for the redesign). I also predict that the printed statements will be identical.

E. Implement and Validate the Redesign (2 marks)

Original and redesigned class compared with the same tests

```
#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>

struct Transaction { char type; double amount; };

// ORIGINAL : fixed array, shift left when full  (cost O(K) per transaction)
template <int K>
class ShiftAccount {
    double balance = 0; Transaction last[K]; int count = 0;
public:
    long long moves = 0;
    void deposit(double a)
    {
        balance += a;
        if (count == K) { for (int i = 1; i < K; ++i) last[i-1] = last[i]; moves += K - 1; --count; }
        last[count++] = {'D', a};
    }
    std::vector<Transaction> snapshot() const { return std::vector<Transaction>(last, last +
    count); }
};

// REDESIGN : circular buffer with a start index (cost O(1) per transaction)
template <int K>
class CircularAccount {
    double balance = 0; Transaction last[K]; int start = 0, count = 0;
public:
    long long moves = 0;                    // always 0 : nothing is ever shifted
    void deposit(double a)
    {
        balance += a;
        int pos = start + count; if (pos >= K) pos -= K;      // avoids '%' on the hot path
        last[pos] = {'D', a};
        if (count == K) { if (++start == K) start = 0; } else ++count;
    }
    std::vector<Transaction> snapshot() const
    {
        std::vector<Transaction> v; v.reserve(count);
        for (int i = 0; i < count; ++i) { int p = start + i; if (p >= K) p -= K; v.push_back(last[p]);
        }
        return v;
    }
};

template <typename Acc>
```

```
double timeIt(long long n, Acc& acc)
{
	double best = 1e18;
	for (int rep = 0; rep < 3; ++rep) {
		auto t0 = std::chrono::steady_clock::now();
		for (long long i = 0; i < n; ++i) acc.deposit((double)(i % 1000));
		auto t1 = std::chrono::steady_clock::now();
		best = std::min(best, std::chrono::duration<double, std::milli>(t1 - t0).count());
	}
	return best;
}
template <int K>
void test(long long n)
{
	ShiftAccount<K> s; CircularAccount<K> c;
	double ts = timeIt(n, s), tc = timeIt(n, c);
	auto a = s.snapshot(), b = c.snapshot();
	bool same = a.size() == b.size();
	for (size_t i = 0; same && i < a.size(); ++i) same = a[i].type == b[i].type && a[i].amount
	== b[i].amount;
	std::cout << "K=" << K << "\t shift: " << ts << " ms\t circular: " << tc
			<< " ms\t speed-up x" << ts / tc << "\t identical statement: " << (same ? "yes" :
			"NO")
			<< "\t moves/txn (shift): " << (double)s.moves / (3.0 * n) << '\n';
}
int main()
{
	const long long N = 2000000;
	std::cout << N << " transactions per run (best of 3)\n";
	test<10>(N); test<50>(N); test<500>(N); test<5000>(N / 10);
}
```

Result at -O2

```
2000000 transactions per run (best of 3)
K=10	shift: 17.9842 ms	circular: 3.50434 ms	speed-up x5.132	identical statement: yes	moves/txn (shift): 8.99999
K=50	shift: 30.3789 ms	circular: 3.8214 ms	speed-up x7.94968	identical statement: yes	moves/txn (shift): 48.9996
K=500	shift: 124.982 ms	circular: 5.5696 ms	speed-up x22.44	identical statement: yes	moves/txn (shift): 498.958
K=5000	shift: 323.129 ms	circular: 0.587437 ms	speed-up x550.066	identical statement: yes	moves/txn (shift): 4957.34
```

Result at -O0

```
2000000 transactions per run (best of 3)
K=10	shift: 49.8388 ms	circular: 13.4209 ms	speed-up x3.71351	identical statement: yes	moves/txn (shift): 8.99999
K=50	shift: 149.621 ms	circular: 9.55065 ms	speed-up x15.6661	identical statement: yes	moves/txn (shift): 48.9996
K=500	shift: 1402.29 ms	circular: 9.21905 ms	speed-up x152.108	identical statement: yes	moves/txn (shift): 498.958
```

K=5000 shift: 1148.36 ms circular: 0.885699 ms speed-up x1296.56 identical statement: yes moves/txn (shift): 4957.34

K	Original (ms) -O2	Redesign (ms) -O2	Speed-up - O2	Speed-up - O0	Same statement
10	18.0	3.5	5.1×	3.7×	yes
50	30.4	3.8	7.9×	15.7×	yes
500	125.0	5.6	22.4×	152×	yes
5000*	323.1	0.59	550×	1297×	yes

* The K = 5000 row uses 200,000 transactions (N/10) to keep the original's run time short, so only the speed-up, not the absolute time, should be compared with the other rows. Each number is the best of 3 runs.

The same tests (the same transaction sequence, the same K, the same compiler flags) were used for both classes, and the final contents of both buffers were compared element by element: they are identical in every run. The per-transaction time of the redesign at -O2 is 1.75 ns for K = 10 and 2.8 ns for K = 500, a growth of 1.6 times, which satisfies the objective of less than 2 times. The speed-up objectives are met: 5.1 times at K = 10 (target 5) and 22.4 times at K = 500 (target 20).

F. Defend the Final Design (2 marks)

- **Was the objective achieved?** Yes. The element moves per transaction dropped from K − 1 to 0, the redesign's time per transaction is almost constant (1.75 to 2.8 ns for K = 10 to 500), the speed-up targets were met at -O2, and the mini-statements are identical.
- **Prediction versus result.** The direction and the growth with K were predicted correctly. The size of the speed-up at -O2 was **lower** than predicted for K = 50 (7.9 times instead of about 10) and K = 500 (22 times instead of more than 50). The reason is that the optimiser turns the shifting loop into a fast block copy, so each moved element costs less than one unoptimised copy. At -O0, where no such optimisation takes place, the speed-ups are much larger (15.7 times and 152 times). The prediction was therefore only partly right and should have taken the optimiser into account.
- **New costs.** One extra int per account, more complex indexing in miniStatement(), and more difficult debugging because the physical order differs from the logical order.
- **Where the original may still be preferable.** For the real requirement of 10 transactions per account the original needs only 9 ns per transaction; an ATM performs a few transactions a day, so the saving is invisible to the user and the simpler, contiguous array (which can be printed or copied in a single loop) is easier to read, test and maintain. The redesign pays off when K is large or when many transactions are recorded per second, for example a server handling millions of accounts.
- **Conclusion.** The circular buffer is the better design whenever K may grow (the bank has already asked for 50), because its cost does not depend on K; the shifting design is acceptable only while K is very small and simplicity is more important.